3.4 Métricas de evaluación

M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF · Gestión 2026-02

En el tema 3.3 usamos el RMSE para comparar modelos sin detenernos a discutirlo. Pero **elegir la métrica es una decisión de diseño**, no un detalle técnico: la métrica define qué significa "funcionar bien", y un modelo optimizado para la métrica equivocada puede ser inútil —o peligroso— en la práctica. En este tema estudiamos las métricas de los dos grandes problemas supervisados: la **regresión**, donde se predice un número, y la **clasificación**, donde se predice una categoría.

> NOTA · **Los datos de este tema**
>
> Para regresión retomamos el modelo lineal del tema 3.3 sobre el dataset **diabetes**, evaluado en sus 89 pacientes de prueba. Para clasificación usamos el dataset **breast cancer** de scikit-learn (`load_breast_cancer`): **569 tumores** descritos por 30 medidas de sus núcleos celulares, de los cuales **212 son malignos** y 357 benignos. Todas las cifras se obtienen con el código que se muestra (`random_state=42`).

3.4.1 Métricas de regresión (MSE, RMSE, R^2)

En regresión, cada predicción $\hat{y}_i$ se compara con el valor real y_i. La diferencia $e_i = y_i - \hat{y}_i$ es el **residuo**. Todas las métricas de regresión son formas distintas de resumir los n residuos en un solo número.

a) Error cuadrático medio (MSE)

$$\text{MSE} = (1/n) \cdot \Sigma_i \, (y_i - \hat{y}_i)^2 \qquad (3.4.1)$$

Es el promedio de los residuos al cuadrado, la misma función de costo que minimizamos en el tema 3.1. Al elevar al cuadrado, los errores positivos y negativos no se cancelan y **los errores grandes pesan mucho más** que los pequeños: un error de 10 aporta 100, cien veces más que un error de 1.

Su principal defecto es la interpretación: el MSE está en **unidades al cuadrado**. Si y se mide en dólares, el MSE está en dólares^2, una magnitud que no tiene significado intuitivo. Por eso el MSE es excelente para optimizar —es derivable y convexo— pero poco útil para comunicar resultados.

b) Raíz del error cuadrático medio (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{[\ (1/n) \cdot \Sigma_i \, (y_i - \hat{y}_i)^2\ ]} \qquad (3.4.2)$$

Sacar la raíz devuelve el error a las **unidades de y**, así que puede leerse como el "error típico" del modelo. Conserva la sensibilidad del MSE a los errores grandes y, como la raíz es una función creciente, ordena los modelos exactamente igual que el MSE. Es la métrica de regresión más usada para informar resultados.

c) Error absoluto medio (MAE)

$$MAE = (1/n) \cdot \Sigma_i \, |y_i - \hat{y}_i| \qquad (3.4.3)$$

El MAE promedia los errores sin elevarlos al cuadrado. También está en las unidades de y, pero trata a todos los errores con el mismo peso relativo, por lo que es **más robusto frente a valores atípicos**. Siempre se cumple $MAE \leq RMSE$, y cuanto mayor es la diferencia entre ambos, más desiguales son los errores: unos pocos errores grandes inflan el RMSE mucho más que el MAE.

EJEMPLO · RMSE frente a MAE con un valor atípico

Dos modelos cometen estos errores absolutos sobre cinco casos:

Modelo A: 2, 2, 2, 2, 2 Modelo B: 1, 1, 1, 1, 6

Modelo A: $MAE = 2.0$ y $RMSE = \sqrt{(20/5)} = 2.0$. **Modelo B:** $MAE = 10/5 = 2.0$ y $RMSE = \sqrt{(40/5)} = \sqrt{8} = 2.83$.

Según el MAE, los dos modelos son igual de buenos. El RMSE, en cambio, castiga al modelo B por su único error grande. ¿Cuál es preferible? Depende del problema: si un error grande es mucho más costoso que varios pequeños (una dosis de medicamento, la carga de un puente), el RMSE refleja mejor el riesgo.

d) Coeficiente de determinación (R^2)

El RMSE tiene un inconveniente: su valor depende de la escala de y. Un RMSE de 50 puede ser excelente o pésimo según el problema. El **R^2** resuelve esto comparando el modelo con el predictor más ingenuo posible, el que predice siempre la media $\bar{y}$:

$$R^2 = 1 - SS_{res} / SS_{tot}$$
$$SS_{res} = \Sigma_i \, (y_i - \hat{y}_i)^2 \qquad SS_{tot} = \Sigma_i \, (y_i - \bar{y})^2 \qquad (3.4.4)$$

SS_{tot} mide la variabilidad total de y; SS_{res}, la variabilidad que el modelo **no** logró explicar. Por eso el R^2 se lee como la **proporción de la varianza de y explicada por el modelo**. No tiene unidades y permite comparar problemas distintos.

Valor de R^2	Significado
$R^2 = 1$	predicción perfecta: todos los residuos son cero
$0 < R^2 < 1$	el modelo explica parte de la variabilidad; mejor cuanto más cerca de 1
$R^2 = 0$	el modelo no es mejor que predecir siempre la media
$R^2 < 0$	el modelo es **peor** que predecir la media

ADVERTENCIA · El R^2 puede ser negativo

Aunque se llame "R al cuadrado", el R^2 **no** es el cuadrado de nada cuando se calcula sobre datos de prueba, y puede ser negativo sin límite. Solo en el conjunto de entrenamiento de una regresión lineal con intercepto está garantizado que $0 \le R^2 \le 1$. El polinomio de grado 3 del tema 3.3, que sobreajustaba, obtiene en prueba $R^2 = -14.56$: sus predicciones son catorce veces peores que usar la media.

e) R^2 ajustado

En el entrenamiento, el R^2 **nunca baja** al agregar variables, aunque sean ruido puro: más columnas siempre permiten ajustar un poco mejor. El **R^2 ajustado** penaliza el número de variables p:

$$R^2_{aj} = 1 - (1 - R^2) \cdot (n - 1) / (n - p - 1) \qquad (3.4.5)$$

Solo aumenta si la nueva variable mejora el ajuste más de lo que cabría esperar por azar. Es habitual en estadística clásica y en informes de modelos lineales; en machine learning, donde la complejidad se controla con validación cruzada, se usa menos.

f) Ejemplo resuelto a mano

EJEMPLO · Métricas de la recta del tema 3.1

Retomemos los cinco puntos del tema 3.1 y la recta ajustada $\hat{y} = 0.6 \cdot x + 2.2$:

y	= 2,	4,	5,	4,	5
$\hat{y}$	= 2.8,	3.4,	4.0,	4.6,	5.2
e	= −0.8,	0.6,	1.0,	−0.6,	−0.2

MSE $= (0.64 + 0.36 + 1.00 + 0.36 + 0.04) / 5 = 2.40 / 5 = 0.48$.

RMSE $= \sqrt{0.48} = 0.693$. **MAE** $= (0.8 + 0.6 + 1.0 + 0.6 + 0.2) / 5 = 3.2 / 5 = 0.64$.

R^2: con $\bar{y} = 4$, $SS_{tot} = 4 + 0 + 1 + 0 + 1 = 6$ y $SS_{res} = 2.40$, de modo que $R^2 = 1 - 2.40/6 = 0.60$. La recta explica el 60 % de la variabilidad de y.

g) Verificación en Python

In [1]:

```
from sklearn.metrics import (mean_squared_error, root_mean_squared_error,
                             mean_absolute_error, r2_score,
                             mean_absolute_percentage_error)

# modelo = pipeline de regresión lineal del tema 3.3 (diabetes)
pred = modelo.predict(X_test)
print(f"MSE  = {mean_squared_error(y_test, pred):.2f}")
print(f"RMSE = {root_mean_squared_error(y_test, pred):.2f}")
print(f"MAE  = {mean_absolute_error(y_test, pred):.2f}")
print(f"R²   = {r2_score(y_test, pred):.3f}")
print(f"MAPE = {mean_absolute_percentage_error(y_test, pred):.1%}")
```

Out [1]:

```
MSE  = 2900.19
RMSE = 53.85
MAE  = 42.79
R²   = 0.453
MAPE = 37.5%
```

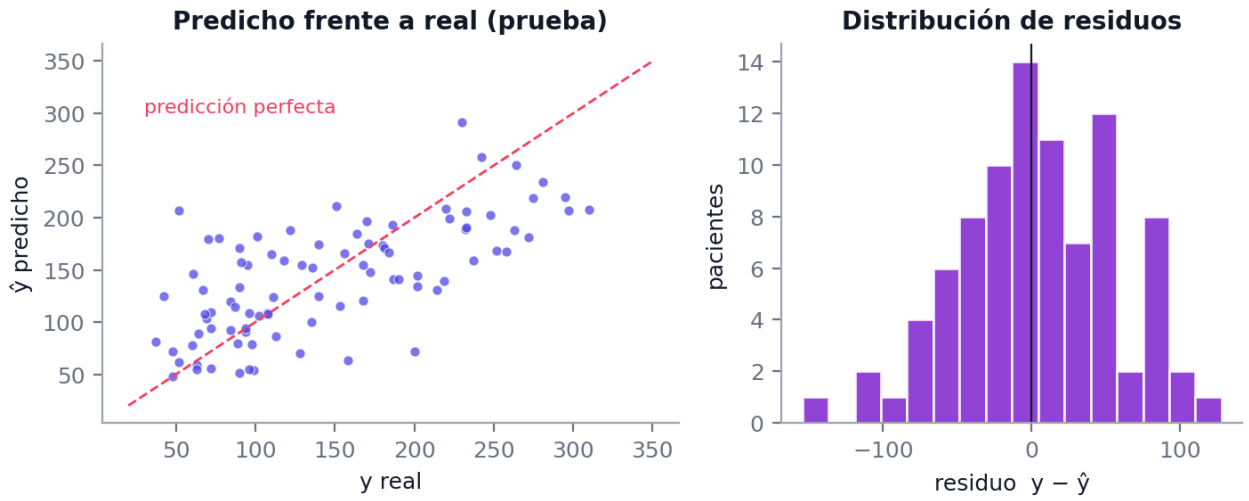


Figura 1. Izquierda: cuanto más cerca de la diagonal, mejor la predicción. Derecha: los residuos se reparten alrededor de cero, con algunos errores superiores a 100.

Lectura de los resultados. El error típico es de unas 54 unidades (RMSE) y el error promedio, de 43 (MAE). La distancia entre ambos indica que hay algunos errores grandes: el mayor llega a 154.5 unidades, mientras que la mitad de los pacientes tiene un error menor de 38. El modelo explica el **45 %** de la varianza de la progresión de la enfermedad; el R^2 ajustado, con 10 variables y 89 pacientes, baja a `0.382`.

> NOTA · **MAPE: el error en porcentaje**
>
> El **error porcentual absoluto medio** $\text{MAPE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i| / |y_i|$ expresa el error como porcentaje del valor real, algo muy fácil de comunicar ("nos equivocamos en un 37.5 % en promedio"). Pero tiene dos problemas serios: se dispara cuando algún y_i está cerca de cero y penaliza más las sobreestimaciones que las subestimaciones. Úsalo solo si y es siempre claramente positiva.

h) ¿Qué métrica de regresión usar?

Métrica	Unidades	Sensible a atípicos	Uso típico
MSE	de y^2	muy alta	función de costo para entrenar
RMSE	de y	alta	informar el error; comparar modelos
MAE	de y	baja	datos con atípicos; costo lineal del error
R^2	sin unidades	alta	proporción explicada; comparar entre problemas
MAPE	%	media	comunicar a no especialistas (con $y > 0$)

> EN MACHINE LEARNING · **Informar siempre más de una métrica**
>
> Ninguna métrica cuenta la historia completa. Una práctica recomendable es informar **RMSE y MAE** (magnitud del error en unidades del problema) junto con $\mathbf{R^2}$ (calidad relativa), y compararlos siempre con el **modelo de referencia**. En el dataset diabetes, predecir la media da `RMSE = 73.22` y `MAE = 64.01`; sin esa referencia, "RMSE = 53.85" no dice si el modelo es bueno o malo.

3.4.2 Métricas de clasificación

En clasificación la salida es una categoría, así que el error ya no es una distancia: cada predicción es simplemente **correcta o incorrecta**. Pero no todos los errores cuestan lo mismo. Diagnosticar como benigno un tumor maligno es mucho más grave que el error contrario, y las métricas de clasificación existen precisamente para distinguir esos tipos de error.

a) La matriz de confusión

En un problema **binario** una de las clases se designa como **positiva** —por convención, la que queremos detectar, aunque sea "mala": el tumor maligno, la transacción fraudulenta, el correo spam— y la otra como **negativa**. Al cruzar la clase real con la predicha surgen cuatro casos:

	Predicho negativo (0)	Predicho positivo (1)
Real negativo (0)	**VN** · verdadero negativo	**FP** · falso positivo (error tipo I)
Real positivo (1)	**FN** · falso negativo (error tipo II)	**VP** · verdadero positivo

La tabla completa se llama **matriz de confusión**, y es el punto de partida de casi todas las métricas. En nuestro ejemplo entrenamos un clasificador de **regresión logística** —un modelo que estudiaremos más adelante; por ahora basta saber que estima la probabilidad de que un tumor sea maligno— y lo evaluamos con los 114 tumores de prueba:



Figura 2. Matriz de confusión del clasificador en el conjunto de prueba: 72 tumores benignos y 42 malignos.

De 114 tumores, el modelo acierta 110. Se equivoca en un tumor benigno que marcó como maligno (**FP = 1**) y en tres malignos que marcó como benignos (**FN = 3**). Esos tres falsos negativos son los errores más graves del problema.

> ADVERTENCIA · **El orden de la matriz en scikit-learn**
>
> `confusion_matrix` pone la **clase real en las filas** y la **predicha en las columnas**, ordenadas de menor a mayor etiqueta: `[[VN, FP], [FN, VP]]`. Otros libros y programas usan el orden inverso, con los positivos primero. Antes de leer una matriz de confusión, comprueba siempre qué hay en cada eje.

b) Exactitud (accuracy)

$$\text{Accuracy} = (VP + VN) / (VP + VN + FP + FN) \qquad (3.4.6)$$

Es la proporción de predicciones correctas: `(39 + 71) / 114 = 0.965`. Es la métrica más intuitiva y funciona bien cuando las clases están **balanceadas** y los dos tipos de error cuestan lo mismo. Fuera de esas condiciones puede engañar gravemente.

> EJEMPLO · **La paradoja de la exactitud**
>
> Un banco revisa 10 000 transacciones, de las cuales **100 son fraudes** (1 %). Un "modelo" que responde siempre *no es fraude* obtiene:
>
> `Accuracy = 9 900 / 10 000 = 0.99`
>
> Un 99 % de exactitud… sin detectar **un solo fraude**. Cuando las clases están desbalanceadas, la exactitud premia a quien ignora la clase minoritaria. Incluso en nuestros datos, que están menos desbalanceados, predecir siempre *benigno* ya da `72 / 114 = 0.632`: esa es la referencia contra la que hay que medir el 0.965.

c) Precision y recall

Las dos métricas siguientes se concentran en la clase positiva y responden preguntas distintas. Usamos los nombres en inglés, que son los habituales; en español se traducen como **precisión** y **exhaustividad** o **sensibilidad**.

$$\text{Precision} = VP / (VP + FP)$$
$$\text{Recall} = VP / (VP + FN) \qquad (3.4.7)$$

Precision responde: *de todos los casos que el modelo marcó como positivos, ¿cuántos lo eran realmente?* Una precision alta significa pocas falsas alarmas. **Recall** responde: *de todos los positivos que existen, ¿cuántos encontró el modelo?* Un recall alto significa que se escapan pocos casos. En medicina, al recall se le llama **sensibilidad**.

Existe además una métrica simétrica para la clase negativa, la **especificidad** `= VN / (VN + FP)`: la proporción de negativos correctamente identificados. Sensibilidad y especificidad son el par de métricas clásico en los estudios clínicos.

Situación	Qué error es más costoso	Métrica a priorizar
Detección de cáncer	no detectar un tumor (FN)	recall
Filtro de spam	enviar a spam un correo legítimo (FP)	precision

Situación	Qué error es más costoso	Métrica a priorizar
Detección de fraude	dejar pasar un fraude (FN)	recall, con precision aceptable
Recomendar productos	mostrar algo irrelevante (FP)	precision
Control de calidad industrial	dejar pasar una pieza defectuosa (FN)	recall

Hay una **tensión** entre ambas. Un modelo que marca todo como positivo alcanza `recall = 1` con una precision pésima; uno que solo marca el caso del que está absolutamente seguro puede tener `precision = 1` con un recall mínimo. Mejorar una suele empeorar la otra.

d) F1-score

Para resumir precision y recall en un solo número se usa su **media armónica**:

$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall) \quad (3.4.8)$$

La media armónica es dura con los desequilibrios: si una de las dos métricas es baja, el F1 también lo es. Con `precision = 1` y `recall = 0.1`, la media aritmética daría `0.55`, pero el F1 es `0.18`. Un F1 alto exige que **ambas** métricas sean altas.

NOTA · **El F-beta: cuando un error importa más**

El F1 da el mismo peso a precision y recall. La generalización $F\beta = (1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$ permite inclinar la balanza: con $\beta = 2$ el recall pesa más (útil en diagnóstico médico) y con $\beta = 0.5$ pesa más la precision (útil en filtros de spam). En scikit-learn: `fbeta_score(y, pred, beta=2)`.

e) Ejemplo resuelto a mano

EJEMPLO · **Métricas a partir de la matriz de confusión**

Con $VN = 71$, $FP = 1$, $FN = 3$ y $VP = 39$ (Figura 2):

Accuracy $= (39 + 71) / 114 = 110 / 114 = 0.965$.

Precision $= 39 / (39 + 1) = 39 / 40 = 0.975$. De cada 40 tumores que el modelo señala como malignos, 39 lo son.

Recall $= 39 / (39 + 3) = 39 / 42 = 0.929$. El modelo encuentra el 92.9 % de los tumores malignos; se le escapan 3 de 42.

Especificidad $= 71 / (71 + 1) = 71 / 72 = 0.986$.

F1 $= 2 \cdot 0.975 \cdot 0.929 / (0.975 + 0.929) = 1.811 / 1.904 = 0.951$.

f) Verificación en Python

In [2]:

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

datos = load_breast_cancer(as_frame=True)
Xc, yc = datos.data, 1 - datos.target     # 1 = maligno (clase positiva)
Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(
    Xc, yc, test_size=0.2, random_state=42, stratify=yc)

clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
clf.fit(Xc_tr, yc_tr)
pred = clf.predict(Xc_te)

print(confusion_matrix(yc_te, pred))
print(classification_report(yc_te, pred, digits=3,
                            target_names=["benigno", "maligno"]))
```

Out [2]:

```
[[71  1]
 [ 3 39]]
              precision    recall  f1-score   support

     benigno      0.959     0.986     0.973        72
     maligno      0.975     0.929     0.951        42

    accuracy                          0.965       114
   macro avg      0.967     0.957     0.962       114
weighted avg      0.965     0.965     0.965       114
```

En scikit-learn, la clase positiva es la etiqueta `1`. El dataset original codifica *maligno* como `0`, por eso invertimos la etiqueta con `1 - datos.target`. El parámetro `stratify=yc` conserva en ambos conjuntos la misma proporción de tumores malignos (37 %).

`classification_report` calcula precision, recall y F1 **para cada clase**, tratándola en turno como positiva: la fila *maligno* reproduce nuestros cálculos a mano; en la fila *benigno*, el recall `0.986` es la especificidad. La columna `support` indica cuántos casos reales hay de cada clase.

> `EN MACHINE LEARNING` · **Promedios en problemas multiclase**
>
> Con más de dos clases, las métricas se calculan para cada clase y luego se promedian. El **promedio macro** (`macro avg`) da el mismo peso a cada clase, de modo que una clase pequeña con mal desempeño se nota. El **promedio ponderado** (`weighted avg`) pondera por el `support`, así que refleja sobre todo a las clases grandes. Con clases desbalanceadas, el promedio macro es el más honesto. En scikit-learn se elige con `average="macro"` o `average="weighted"`.

g) El umbral de decisión

La regresión logística no produce directamente una clase, sino una **probabilidad** `p = P(maligno | x)`. El método `predict` aplica un **umbral** de 0.5: si `p ≥ 0.5`, predice *maligno*. Pero 0.5 es solo una convención. Con `predict_proba` obtenemos las probabilidades y podemos elegir otro umbral según el costo de cada error.

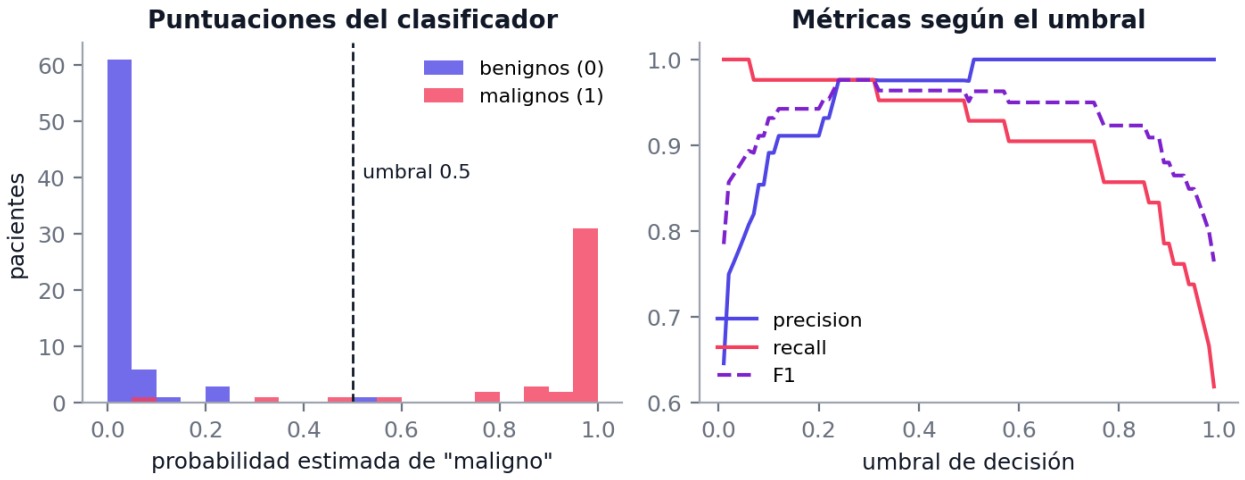


Figura 3. Izquierda: las probabilidades de ambas clases se separan casi por completo. Derecha: al subir el umbral, la precision aumenta y el recall disminuye.

Efecto del umbral en el conjunto de prueba

Umbral	VP	FP	FN	Precision	Recall	F1
0.1	41	5	1	0.891	0.976	0.932
0.3	**41**	**1**	**1**	**0.976**	**0.976**	**0.976**
0.5 (por defecto)	39	1	3	0.975	0.929	0.951
0.7	38	0	4	1.000	0.905	0.950
0.9	33	0	9	1.000	0.786	0.880

Bajar el umbral a 0.3 recupera dos de los tres tumores malignos que se escapaban, sin agregar falsas alarmas. Subirlo a 0.9 elimina todos los falsos positivos, pero deja sin detectar 9 de 42 tumores malignos: inaceptable en un contexto médico. **El umbral es una decisión del negocio o del dominio**, no del algoritmo.

> ADVERTENCIA · **El umbral también se elige con validación**
>
> En la tabla anterior el umbral 0.3 parece el mejor, pero lo descubrimos **mirando el conjunto de prueba**. Como vimos en el tema 3.3, elegir cualquier cosa con los datos de prueba vuelve optimista la evaluación. En un proyecto real, el umbral se elige con validación cruzada sobre el entrenamiento (por ejemplo, con `TunedThresholdClassifierCV`) y luego se evalúa una sola vez en prueba.

h) Curva ROC y AUC

Cada umbral produce una matriz de confusión distinta. La **curva ROC** (*Receiver Operating Characteristic*) resume todos los umbrales a la vez: grafica la **tasa de verdaderos positivos** (el recall) frente a la **tasa de falsos positivos** para cada umbral posible:

$$\text{TPR} = \text{VP} / (\text{VP} + \text{FN})$$
$$\text{FPR} = \text{FP} / (\text{FP} + \text{VN}) = 1 - \text{especificidad} \qquad (3.4.9)$$

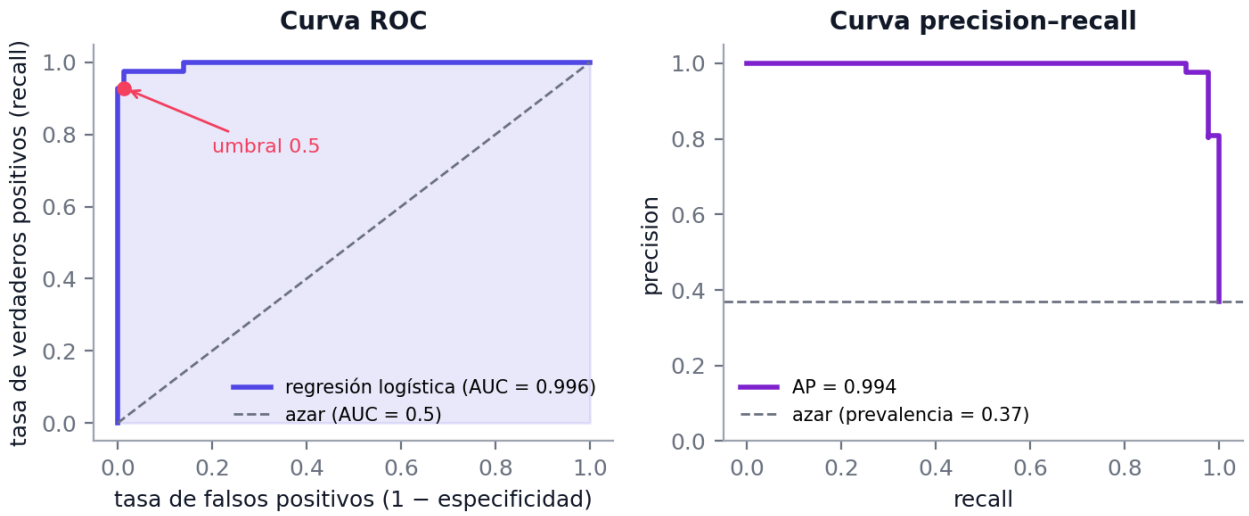


Figura 4. Curva ROC (izquierda) y curva precision–recall (derecha) del clasificador. El punto coral es el umbral 0.5.

Con un umbral de 1, nada se marca como positivo y la curva empieza en `(0, 0)`; con un umbral de 0, todo es positivo y termina en `(1, 1)`. Un clasificador perfecto pasa por la esquina superior izquierda `(0, 1)`: detecta todos los positivos sin ninguna falsa alarma. Un clasificador que adivina al azar sigue la diagonal.

El **AUC** (*Area Under the Curve*) es el área bajo la curva ROC. Vale 1 para un clasificador perfecto y 0.5 para uno aleatorio. Nuestro modelo alcanza `AUC = 0.996`. El AUC tiene una interpretación muy clara:

> **DEFINICIÓN · Interpretación probabilística del AUC**
>
> El AUC es la **probabilidad de que el modelo asigne una puntuación mayor a un caso positivo elegido al azar que a un caso negativo elegido al azar**. Un `AUC = 0.996` significa que, si tomamos un tumor maligno y uno benigno al azar, en el 99.6 % de los casos el modelo le dará mayor probabilidad de malignidad al que realmente lo es.

Dos propiedades hacen útil al AUC: **no depende del umbral**, porque evalúa la capacidad del modelo para *ordenar* los casos, y **no depende de la proporción de clases**. Por eso es la métrica preferida para comparar clasificadores antes de fijar un umbral.

i) Curva precision–recall

Cuando la clase positiva es **muy minoritaria**, la curva ROC puede ser engañosamente optimista: como hay muchísimos negativos, incluso cientos de falsos positivos producen una tasa de falsos positivos pequeña. La **curva precision–recall** (Figura 4, derecha) no usa los verdaderos negativos, así que refleja mejor el desempeño sobre la clase rara. Se resume con la **precisión promedio** (*average precision*, AP), que en nuestro modelo vale `0.994`. A diferencia de la ROC, su línea de referencia no es 0.5 sino la **prevalencia** de la clase positiva, `42 / 114 = 0.37`.

In [3]:

```
from sklearn.metrics import roc_auc_score, average_precision_score

proba = clf.predict_proba(Xc_te)[:, 1]    # probabilidad de "maligno"
print(f"AUC ROC = {roc_auc_score(yc_te, proba):.3f}")
print(f"AP      = {average_precision_score(yc_te, proba):.3f}")

pred_03 = (proba >= 0.3).astype(int)      # umbral personalizado
print(confusion_matrix(yc_te, pred_03))
```

Out [3]:

```
AUC ROC = 0.996
AP      = 0.994
[[71  1]
 [ 1 41]]
```

Observa que `roc_auc_score` y `average_precision_score` reciben las **probabilidades**, no las clases predichas: necesitan conocer el orden de las puntuaciones para recorrer todos los umbrales. Pasarles el resultado de `predict` es un error frecuente: con solo dos valores posibles se evalúa un único umbral y el resultado ya no es el AUC del modelo.

j) ¿Qué métrica de clasificación usar?

Situación	Métrica recomendada
Clases balanceadas y errores de costo similar	accuracy
Los falsos negativos son costosos	recall (sensibilidad), F2
Los falsos positivos son costosos	precision, F0.5
Se busca equilibrio en la clase positiva	F1

Situación	Métrica recomendada
Comparar modelos antes de fijar un umbral	AUC ROC
Clase positiva muy rara (fraude, fallas)	curva precision–recall, AP
Varias clases desbalanceadas	F1 con promedio macro
Siempre	la matriz de confusión completa

> NOTA · **Síntesis del tema**
>
> En **regresión**, el **MSE** es la función que se optimiza, el **RMSE** y el **MAE** expresan el error en las unidades del problema —el primero más sensible a los errores grandes— y el $\mathbf{R^2}$ mide la proporción de varianza explicada, que puede ser negativa si el modelo es peor que la media. En **clasificación**, todo parte de la **matriz de confusión**: la **accuracy** engaña con clases desbalanceadas; **precision** y **recall** separan las falsas alarmas de los casos no detectados; el **F1** los combina; el **umbral** traslada el costo de cada error a la decisión; y la curva **ROC** con su **AUC** evalúa la capacidad del modelo para ordenar los casos, independientemente del umbral. En todos los casos, la métrica se compara con un **modelo de referencia**.